

Part 5: WPA2 offline candidate verification

Ethical and authorized use: use only the supplied course PCAP and synthetic metadata. Do not capture traffic, deauthenticate clients, disrupt networks, use a wireless interface, or test third-party credentials. This exercise begins after an authorized capture has already been provided.

The password is not transmitted in a WPA2 four-way handshake. Nevertheless, a recorded SSID, AP/STA MAC addresses, ANonce/SNonce, EAPOL bytes, and MIC let a candidate be checked offline:

```
candidate passphrase + SSID -> PBKDF2-HMAC-SHA1 -> PMK
PMK + ordered MACs/nonces -> PRF -> PTK -> KCK
KCK + EAPOL frame -> candidate MIC -> compare with captured MIC
```

Run the explanation and single-candidate checker against `../data/wpa2_capture.json`. Server-side rate limiting cannot observe or slow local guesses after capture. WPA2-Enterprise and WPA3-SAE have different authentication properties and are outside this exercise.

Purpose and learning objectives

This part connects password guessing to an authentication protocol. You are given both a previously captured course PCAP and a small synthetic metadata fixture. You will not capture packets. First identify the four-way handshake, then study the derivation calculation, and finally run a bounded dictionary check against the supplied capture.

After completing this part, you should be able to:

- identify AP, STA, SSID, nonces, replay counters, and EAPOL-Key messages;
- explain how WPA2-Personal derives PMK, PTK, and KCK;
- describe what M1, M2, M3, and M4 accomplish;
- explain how a MIC confirms a candidate without transmitting the password;
- classify the dictionary check as offline; and
- state the authorization boundary for wireless-security experiments.

Files used

File	Purpose
---	---
<code>../data/wpa2/lab01-handshake.pcap</code>	Supplied course packet capture
<code>../data/wpa2/lab01-pcap-wordlist.txt</code>	Small candidate list for that PCAP only
<code>../data/wpa2_capture.json</code>	Synthetic values for transparent Python calculations
<code>inspect_capture.py</code>	Summarizes networks and EAPOL-Key frames
<code>explain_handshake.py</code>	Labels the synthetic captured values and derivation chain

Tasks

1. Run inspect_capture.py and confirm the target BSSID, SSID, and M1-M4 order.
2. For every message, record the sender, receiver, and replay counter.
3. Read the key-derivation explanation below and label PMK, PTK, KCK, KEK, TK, ANonce, SNonce, and MIC in your own diagram.
4. Run explain_handshake.py on the synthetic JSON and connect each printed field to the derivation formula.
5. Run verify_candidate.py with one nonmatching candidate from the course list; explain why no match is useful evidence.
6. Use aircrack-ng with exactly the provided PCAP, wordlist, and BSSID.
7. Record that a key was found, tested-key statistics, and timing, but do not publish the recovered key.
8. Explain why no server, AP, or wireless interface receives the guesses.

How the WPA2 four-way handshake establishes keys

Despite its common name, the four-way handshake does not transmit the Wi-Fi passphrase or directly exchange the final encryption key. Before the handshake, both sides must already be able to obtain the same Pairwise Master Key (PMK). For WPA2-Personal, the passphrase and SSID produce the PMK:

```
PMK = PBKDF2-HMAC-SHA1(  
    passphrase,  
    SSID,  
    iterations = 4096,  
    output length = 32 bytes  
)
```

The access point (AP) and station/client (STA) then contribute fresh nonces. Both sides combine the PMK with the ordered MAC addresses and nonces to derive the same session-specific Pairwise Transient Key (PTK):

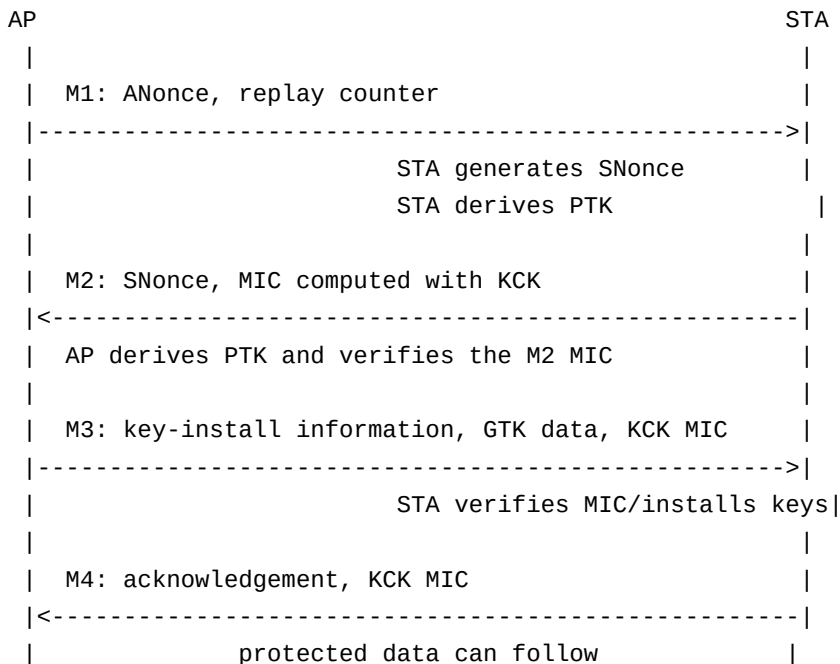
```
context = min(AP_MAC, STA_MAC) || max(AP_MAC, STA_MAC)  
         || min(ANonce, SNonce) || max(ANonce, SNonce)  
  
PTK = PRF-512(PMK, "Pairwise key expansion", context)
```

The min/max notation means lexicographic byte ordering. It ensures that the AP and STA build exactly the same context even though each views itself as the local endpoint.

The PTK is divided into keys with separate purposes:

- KCK (Key Confirmation Key): authenticates handshake messages by computing their MICs;
- KEK (Key Encryption Key): protects key material transported in EAPOL-Key messages; and
- TK (Temporal Key): protects later unicast data traffic.

The four EAPOL-Key messages have the following simplified roles:



- M1 gives the STA the AP-generated ANonce. It normally has no MIC because the STA does not yet have all PTK inputs.
- M2 returns the STA-generated SNonce and proves that the STA derived a KCK consistent with the shared PMK.
- M3 proves the AP derived the same keys and tells the STA to install them; it can also carry group-key information protected with the KEK.
- M4 acknowledges successful installation and completes the exchange.

Replay counters help reject reused handshake messages. Nonces make the PTK specific to this session, even when the same network passphrase is reused.

Why the capture permits an offline guess

The capture exposes the SSID, AP/STA MAC addresses, ANonce, SNonce, an EAPOL message, and its MIC. For each dictionary candidate, an offline program can:

1. derive a candidate PMK from the candidate and captured SSID;
2. derive a candidate PTK from the captured addresses and nonces;
3. take the KCK from the candidate PTK;
4. zero the MIC field in the captured EAPOL bytes and recompute the MIC; and

5. compare the candidate MIC with the captured MIC.

A match is evidence that the candidate generated the same key material. No guess is sent to the AP, so the AP cannot apply server-side rate limiting. The handshake still does not reveal a general shortcut: the attacker must test candidates, which is why passphrase strength matters.

Python files and usage examples

`'inspect_capture.py'`

Inspect the supplied packet capture without connecting to any network:

```
cd /workspace/labs/lab01/part5_wpa2
python3 inspect_capture.py ../data/wpa2/lab01-handshake.pcap
```

Expected summary:

```
Packets: <count>
Network: SSID=Coherer BSSID=00:0c:41:82:b2:55
EAPOL-Key frames: 4
M1: <sender> -> <receiver> (replay=0)
M2: <sender> -> <receiver> (replay=0)
M3: <sender> -> <receiver> (replay=1)
M4: <sender> -> <receiver> (replay=1)
```

The M1-M4 sequence is evidence that the file contains the values needed for an offline WPA/WPA2-Personal candidate check. This parser summarizes only the course PCAP; it does not implement password cracking.

`'explain_handshake.py'`

```
cd /workspace/labs/lab01/part5_wpa2
python3 explain_handshake.py ../data/wpa2_capture.json
```

Expected output lists the synthetic SSID, AP and client MAC addresses, two nonces, and the captured MIC, followed by the PMK -> PTK -> KCK -> MIC chain. The JSON file is calculation metadata created for the course, not captured third-party traffic.

To see its required local-file argument, run:

```
python3 explain_handshake.py --help
```

`'verify_candidate.py'`

The candidate is read with `getpass`, so it is not displayed or stored in shell history:

```
python3 verify_candidate.py ../data/wpa2_capture.json
Candidate:
no match
```

A candidate consistent with the synthetic record prints match; other inputs print no match. Try candidates only from the provided classroom wordlist. The script performs no network or radio operation.

The help output confirms that the only argument is a local metadata file:

```
python3 verify_candidate.py --help
```

Offline dictionary check of the supplied PCAP

Use `aircrack-ng` only with the two local course files:

```
aircrack-ng \
  -w ../data/wpa2/lab01-pcap-wordlist.txt \
  -b 00:0c:41:82:b2:55 \
  ../data/wpa2/lab01-handshake.pcap
```

The `-b` argument selects the Coherer access point found during inspection, so the command cannot accidentally choose another network in the file. The tool should report that it found a key from the small instructor-provided list. Record the number of tested keys and elapsed time in your report, but do not publish the recovered key. The password was not read from a packet: each candidate was used to derive key material and reproduce the captured MIC.

Do not substitute a downloaded wordlist or another capture. This is an offline classroom demonstration, not a procedure for acquiring wireless traffic.

Completion criteria and report evidence

You have completed Part 5 when the inspector reports the expected target and all four handshake messages, you can explain the derivation chain, the synthetic checker distinguishes a matching from a nonmatching candidate, and aircrack-ng reports a result for the supplied course files. Include the M1-M4 summary, your own key-derivation diagram, commands, redacted result/timing, and authorization statement. Never include the recovered passphrase in a public report or screenshot.

Checkpoint questions

1. Which captured values make candidate verification possible?
2. Why is this classified as offline guessing?
3. Where is the password used even though it never crosses the network?
4. Why can server-side rate limiting not slow this computation?
5. Why should these WPA2-Personal observations not be generalized directly to WPA2-Enterprise or WPA3-SAE?
6. Which four EAPOL-Key messages does inspect_capture.py find, and which side transmits each one?